



Nova: A Two-Tier Metastable Finality Ladder for Lux

Majority-accept and $\frac{2}{3}$ -stake export,
decomposed — the v1.36 consensus

Zach Kelling

Lux Industries

2026-07-10

Abstract

Lux v1.36 (consensus v1.36.0, node v1.36.0, evm v1.104.8) ships *Nova*: a leaderless, post-quantum, metastable consensus whose defining structural choice is a **two-tier finality ladder**. A block climbs five distinct authorization rungs — **Photon** → **Wave** → **Nova** → **Quasar** → **Horizon** — and the two load-bearing rungs are kept *orthogonal*. **Nova** is *local acceptance* at a simple majority of the validator set, $\text{NovaQuorum}(n) = \lfloor n/2 \rfloor + 1$; it is crash-fault-safe and authorizes local execution but is reorgable. **Quasar** is *export finality*: a strict $> \frac{2}{3}$ -of-stake, Byzantine-safe post-quantum certificate that alone authorizes bridges, DEX settlement, and cross-chain messages. The invariant is one line: *a block below Quasar can never be exported*. This separation — majority to make local progress, supermajority to make irreversible external claims — is why a five-validator fleet keeps producing at three live nodes (where a single $\frac{2}{3}$ -only engine freezes) while never exporting an unsafe block.

We specify the ladder against the shipped code (`engine/chain/finality.go`, `quorum.go`, `cert.go`, `config/quorum_threshold.go`), give the safety arguments (crash-fault majority-intersection for Nova; the Byzantine overlap bound $2\alpha - K > f$ and a no-two-conflicting-certificates theorem for Quasar), describe the ZAP-native plugin transport (`luxfi/zap`, no gRPC, no BFT round machine), and show the rule is size-invariant from $n = 1$ to $n = 10^6$ because K is a *sample* size, not the set size. We then report measured evidence: a live 5-validator testnet (`ghcr.io/luxfi/node:v1.36.0`) held common-height coherence through a $5/5 \rightarrow 4/5 \rightarrow 3/5$ node-drop while producing (+20, +8, +6 blocks per 70 s phase), and a steady load run measured the live C-Chain mining at $\sim 1.6\%$ block fullness with the base fee pinned at its floor — locating the live throughput ceiling in the EVM fee/gas-target throttle, *not* in consensus, which stayed coherent and idle. Every number is tagged by provenance (**M** measured-now, **M*** measured-prior, **P** projected, **C** cited).

Contents

1	Introduction: one ladder, five rungs, two tiers that matter	4
2	Nova — majority local-acceptance	5
2.1	The quorum: $\text{NovaQuorum}(n) = \lfloor n/2 \rfloor + 1$	5
2.2	The confidence depth β	5
2.3	Fail-closed, never fork-open	5
3	Quasar — $\frac{2}{3}$ -stake export finality	6
3.1	The stake predicate: one definition, strict $> \frac{2}{3}$	6
3.2	The certificate carries its tier	6
3.3	Trails acceptance; gates only export	7
3.4	Post-quantum composition	7
4	The two-tier separation as a safety theorem	7
4.1	Quasar's Byzantine overlap bound	8
4.2	Why the reduction is clean	8
4.3	Where Lux sits among the thresholds	8
5	ZAP-native transport — no gRPC, no round machine	9
6	Size-invariance: the same rule from $n = 1$ to $n = 10^6$	9
7	Empirical evaluation	9
7.1	Coherence under a node-drop — the two-tier property, live	10
7.2	Live C-Chain throughput — the ceiling is the fee/gas-target, not consensus	10
7.3	Relationship to execution — Block-STM is downstream and (today) sequential	11
7.4	Consensus engine bookkeeping is not the bottleneck	11

8	Terminology reconciliation and relationship to prior LPs	11
9	Conclusion	12

1 Introduction: one ladder, five rungs, two tiers that matter

A consensus stack answers one hard question — *when is a block irreversible?* — and a well-built stack answers it in exactly one place. Lux answers it with a **finality ladder**: an ordered set of authorization tiers, each a strictly higher authority than the last, with the boundaries between them never collapsed. The ladder is named in the Lux astrophysics ontology — light is emitted, it propagates, a star ignites, it blazes into the brightest beacon in the sky, it crosses the event horizon:

Photon → Wave → Nova → Quasar → Horizon

Rung	Meaning	Authorizes
Photon	a vote is in flight; the block is being sampled	nothing
Wave	the sampling wave has made this block the build tip	nothing
Nova	β -ignition: a <i>majority</i> sustained β rounds	local execution (reorgable)
Quasar	a strict $> \frac{2}{3}$ -stake certificate exists	export : bridges / DEX / cross-chain
Horizon	a post-quantum signature seals the certificate	irreversible settlement

The two rungs that carry the protocol’s weight are **Nova** and **Quasar**, and the central design decision of v1.36 is that *they are different tiers with different thresholds and different jobs*:

- **Nova** is *local acceptance*. It fires at a *simple majority* of the validator set. It is crash-fault-safe — it survives any minority going down — and it authorizes the node to execute the block locally. It is *reorgable*: a lone equivocator can, in principle, split a bare majority, so Nova is safe to *act on* but never to *export*.
- **Quasar** is *export finality*. It fires only when a strict $> \frac{2}{3}$ of *stake* has signed — the Byzantine-safe threshold — and it is the *only* tier that may drive an irreversible external effect: a bridge withdrawal, a 0x9999 DEX settlement receipt, a cross-chain message, a validator-set transition.

THE INVARIANT. A block below Quasar can *never* be exported as deterministically final to a bridge, another chain, or an irreversible settlement. Nova authorizes local execution; export requires Quasar; quantum-irreversibility requires Horizon. (engine/chain/finality.go.)

This paper is the rigorous, measured treatment of that ladder. It is the companion to the normative specification **LP-305** [1] and the cross-system landscape **LP-306** [2]; where this paper and the code disagree, the code (consensus v1.36.0) governs and this paper is the defect.

Why two tiers and not one. A single threshold cannot be both maximally live and maximally safe. Set the bar at a majority and you get liveness down to $n/2$ but you cannot make a Byzantine-safe external claim. Set it at $\frac{2}{3}$ of stake and you get a Byzantine-safe claim but you freeze the moment more than a third is unreachable — a five-node fleet stops at three live nodes. Nova refuses the false choice: it runs *both* thresholds, on *different rungs*, for *different purposes*. The majority tier keeps the chain making local progress; the supermajority tier gates only what leaves the chain. A node that has lost its supermajority keeps accepting and executing (Nova) while *honestly* declining to export (no Quasar) until the supermajority returns.

Provenance discipline. This is a measurement paper; it does not launder projections as results. Every quantitative claim is tagged: **M** = measured by us this session (2026-07-10) on the hardware named in §7; **M*** = measured in prior cited Lux work; **P** = projected or derived by calculation; **C** = an external cited claim. Untagged numbers are definitions.

2 Nova — majority local-acceptance

Nova is the *lower* of the two load-bearing tiers and the one that carries chain liveness. It is driven by a leaderless metastable sampler (Photon committee sampling → Wave / FPC threshold voting → Focus β -confidence; LP-111 / LP-027 / LP-028 / LP-114), and it *ignites* — transitions a block to the Nova rung and calls `VM.Accept` — when a majority of the validator set has sustained the block for β consecutive rounds.

2.1 The quorum: $\text{NovaQuorum}(n) = \lfloor n/2 \rfloor + 1$

The Nova ignition threshold is a clean function of the live validator count n (`engine/chain/quorum.go`):

```
1 // NovaQuorum is the majority the sampler needs to IGNITE a block to Nova
2 // (acceptance, local execution). It tolerates floor((n-1)/2) crash faults --
3 // the MOST a net of n can lose and still make safe local progress under the
4 // crash-fault model: any two majorities intersect, a non-equivocating node
5 // cannot back two conflicting blocks, and a partition lets at most one side
6 // hold a majority. It is DELIBERATELY below the Quasar 2/3 stake gate.
7 func NovaQuorum(n int) int {
8     if n < 1 { return 1 } // a lone node self-ignites; never 0
9     return n/2 + 1
10 }
11
12 // CrashTolerance = n - NovaQuorum(n) = ceil(n/2) - 1: crash faults Nova
13 // ignition survives. Beyond it Nova PAUSES fail-closed (no ignition => no
14 // acceptance => no fork) and resumes the instant a node returns.
15 func CrashTolerance(n int) int {
16     if n < 2 { return 0 }
17     return n - NovaQuorum(n)
18 }
```

Definition 2.1 (Nova quorum and crash tolerance). *For a validator set of size n , $\text{NovaQuorum}(n) = \lfloor n/2 \rfloor + 1$ and $\text{CrashTolerance}(n) = n - \text{NovaQuorum}(n) = \lceil n/2 \rceil - 1$.*

For the live $n = 5$ primary-network fleet this is $\text{NovaQuorum}(5) = 3$ and $\text{CrashTolerance}(5) = 2$: **Nova keeps accepting with any three of five nodes responsive.** This single fact is the operational heart of v1.36 — and it is exactly where a $\frac{2}{3}$ -only engine dies, because $\lceil 2 \cdot 5/3 \rceil = 4 > 3$.

2.2 The confidence depth β

Ignition additionally requires β *consecutive* majority rounds (hysteresis), so a one-round flap cannot ignite (`quorum.go`):

```
1 // NovaBeta: n==1 -> 1 (a lone node ignites immediately; any wait is a
2 // phantom-peer stall). 2<=n -> 2 (one confirming round of hysteresis, without
3 // the fragility of a deep beta at zero margin). On K=n nets beta is NOT a
4 // Byzantine knob -- that is Quasar's job; large permissionless nets (K<n, real
5 // subsampling) raise beta to the classical negl(beta) metastable depth.
6 func NovaBeta(n int) int { if n <= 1 { return 1 }; return 2 }
```

2.3 Fail-closed, never fork-open

When responsive stake drops below $\text{NovaQuorum}(n)$, Nova does not lower its bar and it does not guess — it **pauses**. No ignition means no acceptance means no fork; the chain halts safely and resumes, via bounded event-driven re-poll and runtime ancestor catch-up (LP-303 [3]), the instant a node returns. A paused Lux chain is a recoverable operational event; it is never a divergent chain. This is the crash-fault contract: *make progress at a majority, or make no unsafe move at all.*

Theorem 2.1 (Nova crash-fault safety). *Under the crash-fault model (validators may halt but do not equivocate), if $\text{NovaQuorum}(n) = \lfloor n/2 \rfloor + 1$ distinct validators must accept a block at a height before it is Nova-final, then no two conflicting blocks are ever both Nova-final at that height.*

Proof. Let B and B' be Nova-final at the same height, certified by acceptor sets S, S' with $|S|, |S'| \geq \lfloor n/2 \rfloor + 1$. Then $|S \cap S'| \geq |S| + |S'| - n \geq 2(\lfloor n/2 \rfloor + 1) - n \geq 1$, so some validator $v \in S \cap S'$ accepted both. Under the crash-fault model a validator never accepts two conflicting blocks at one height (it accepts, then halts or continues, but does not equivocate), so $B = B'$. A network partition cannot manufacture two majorities: at most one side of any partition can hold $\lfloor n/2 \rfloor + 1$ of n . $\square$

Theorem 2.1 is exactly why Nova is safe to *act on locally* but is still labeled *reorgable*: the crash-fault guarantee does not cover a genuine equivocator, who could sign for two blocks and split a bare majority. Guarding against that adversary is not Nova’s job — it is Quasar’s.

3 Quasar — $\frac{2}{3}$ -stake export finality

Quasar is the *upper* load-bearing tier. It produces a portable, post-quantum certificate that a strict supermajority of *stake* attested to a block Nova already accepted. It is the only tier the invariant permits to leave the chain.

3.1 The stake predicate: one definition, strict $> \frac{2}{3}$

The supermajority floor has exactly one definition (`config/quorum_threshold.go`), overflow-safe, shared by both the certificate verifier and the live-set parameter sizer so the two can never drift:

```

1 // TwoThirdsStakeFloor(total) = floor(2*total/3), overflow-safe.
2 // A Quasar certificate is valid iff signed stake is STRICTLY greater.
3 func TwoThirdsStakeFloor(total uint64) uint64 {
4     q, r := total/3, total%3
5     floor := 2 * q
6     if r == 2 { floor++ }           // floor(2r/3): r in {0,1}->0, r==2->1
7     return floor
8 }
9 // Equal-stake vote-count form: floor(2n/3)+1.
10 func EqualStakeSupermajorityThreshold(n int) int {
11     if n <= 0 { return 1 }
12     return int(TwoThirdsStakeFloor(uint64(n))) + 1
13 }

```

3.2 The certificate carries its tier

A `QuorumCert` is tagged with the rung it certifies, and verification dispatches on that tag — the two-tier ladder is not a convention layered over the wire, it *is* the wire (`engine/chain/cert.go`):

```

1 func (c *QuorumCert) VerifyWeighted(v VoteVerifier, stake StakeSource, epochHeight
   uint64) error {
2     if err := c.Verify(v, epochHeight); err != nil { return err }
3     switch c.Tier {
4     case Nova: return c.verifyNovaMajority(stake, epochHeight)           // >= floor(n/2)
5         +1 voters
6     case Quasar: return c.verifyQuasarSupermajority(stake, epochHeight) // > floor(2/3 *
7         stake)
8     default: return fmt.Errorf("%w: tier=%s", ErrQCUnknownTier, c.Tier)
9     }
10 }
11 func (c *QuorumCert) verifyQuasarSupermajority(stake StakeSource, epochHeight uint64)
   error {

```

```

11 total := stake.TotalStake(epochHeight) // read at the P-CHAIN EPOCH height
12 if total == 0 { return ...fail-closed... }
13 var voted uint64
14 for i := range c.Votes { voted += stake.Weight(c.Votes[i].NodeID, epochHeight) }
15 if voted <= config.TwoThirdsStakeFloor(total) { return ...below supermajority... }
16 return nil // STRICT > floor(2/3 * total)
17 }

```

Three details make this correct rather than merely plausible:

1. **Strict inequality.** Acceptance is $\text{voted} > \lfloor \frac{2}{3} \text{total} \rfloor$ (Tendermint $+\frac{2}{3}$), never $\geq$.
2. **Read at the epoch height.** The stake tally is read at the block's P-chain *epoch* height — the same height the certificate's set-root and per-voter public keys are read at — not the value-chain height, which races ahead of the P-chain and which the platform VM would reject as unfinalized. This binds every counted vote to the weight the signer actually held when it signed.
3. **One floor everywhere.** `verifyQuasarSupermajority` and the live-set α -sizer both call `config.TwoThirdsStakeFloor`, so the count threshold a node sizes to can never disagree with the stake predicate a verifier enforces.

3.3 Trails acceptance; gates only export

Quasar runs *strictly after VM.Accept*. Each validator, *having already accepted* block B , signs the canonical finality message for B (which binds the inner canonical execution identity and the epoch set-root and excludes the outer proposer-VM envelope); a collector aggregates signatures bucketed by (height, inner-id) and emits one certificate when the bucket clears the stake floor. Consequently:

- An absent, late, or never-formed Quasar certificate **cannot** stall the chain, revert a block, or change which block Nova accepted. Quasar *describes* an outcome; it does not *decide* one.
- What Quasar gates is *export*, not *progress*. A node below its stake supermajority keeps climbing to Nova and executing locally; it simply does not emit the export certificate. This is the honest degraded mode, not a failure: local progress continues, external claims wait.
- The signed bytes are the unchanged `CanonicalVoteMessage`, the certificate is the unchanged `QuorumCert`, and external verification is the unchanged `VerifyWeighted`; bridges, 0x9999 DEX receipts, and light clients are unaffected by anything internal to the engine (LP-182, LP-217).

3.4 Post-quantum composition

The certificate's signature legs are the Lux PQ suite, selected by the operator's cert profile (LP-217): PQ-off (BLS only), PQ-fast (BLS || Pulsar), PQ-strict (BLS || Pulsar || Corona, main-net default), PQ-heavy (adds Magnetar SLH-DSA). Pulsar contributes Module-LWE threshold signatures byte-equal to FIPS-204 ML-DSA; Magnetar contributes hash-based SLH-DSA; the Horizon rung seals the certificate so that it is irreversible even against a quantum adversary. Because Quasar is post-accept, **the post-quantum cost is off the critical finality path** — the chain has already advanced to Nova when the certificate is being assembled.

4 The two-tier separation as a safety theorem

The value of separating the tiers is that each carries exactly one obligation and the two compose without a seam.

4.1 Quasar’s Byzantine overlap bound

The metastable sampler is configured so that two acceptance quorums must overlap in *more than* f honest nodes, where $f = \lfloor (K - 1)/3 \rfloor$. This is the BFT-overlap bound enforced in `config.Parameters.Valid()`:

$$2\alpha - K \geq \lfloor \frac{K-1}{3} \rfloor + 1 = f + 1,$$

i.e. $2\alpha - K > f$. Configurations that fail it (e.g. $K=3, \alpha=2$ posing as BFT) are rejected with `ErrAlphaBelowBFTQuorum`; mainnet additionally requires $K \geq 11$ so $f \geq 3$, and a value/PoS chain requires $K \geq 4$. The node *fails closed*: it refuses to start a multi-node value chain with non-BFT parameters rather than run an unsafe quorum.

Theorem 4.1 (No two conflicting Quasar certificates). *Assume honest validators sign a finality message only for a block they accepted, Nova accepts at most one block per height (Theorem 2.1, w.h.p. under the sampler margin), and Byzantine stake is $< \frac{1}{3}$ of total. Then no two Quasar certificates for conflicting blocks $B \neq B'$ at the same height can both verify.*

Proof. A valid Quasar certificate for B carries signed stake $w > \lfloor \frac{2}{3}T \rfloor$, so strictly more than $\frac{2}{3}$ of total stake T . Two such certificates for $B \neq B'$ would together require $> \frac{4}{3}T$ signed stake, exceeding T by $> \frac{1}{3}T$; by inclusion–exclusion the overlap carries $> \frac{1}{3}T$ of stake that signed *both*. Since Byzantine stake is $< \frac{1}{3}T$, the overlap contains honest stake, and an honest validator signs only what it accepted, so it signed at most one of B, B' — contradiction. Hence at most one conflicting certificate verifies. $\square$

4.2 Why the reduction is clean

Quasar adds *no* new safety assumption and *no* liveness dependency beyond what the legacy $\frac{2}{3}$ -stake certificate already carried. Its safety reduces to Nova’s single-accept-per-height (Theorem 2.1) plus the same $\geq \frac{2}{3}$ -honest- stake assumption; nothing waits on it. The composition is therefore:

$$\underbrace{\text{Nova (majority)}}_{\text{liveness to } n/2, \text{ crash-safe local accept}} \prec \underbrace{\text{Quasar } (\frac{2}{3} \text{ stake})}_{\text{Byzantine-safe export}} \prec \underbrace{\text{Horizon (PQ seal)}}_{\text{quantum-irreversible}}.$$

Each rung is independently complete; each authorizes strictly more than the last; and the invariant — nothing below Quasar exports — is a single comparison, `finality  $\geq$  Quasar`, checked at every export boundary (`Finality.AuthorizesExport`).

4.3 Where Lux sits among the thresholds

Family	Safety threshold	Finality
Lux Nova (local)	majority $\lfloor n/2 \rfloor + 1$, crash-safe	sub-second, reorgable
Lux Quasar (export)	strict $> \frac{2}{3}$ stake, Byzantine	deterministic PQ certificate
Classical BFT (Tendermint, HotStuff)	$\frac{1}{3}$ of a fixed committee	instant, but $O(n^2)$, does not scale
Nakamoto (Bitcoin)	$> 50\%$ hashpower	probabilistic, minutes

Lux keeps the metastable sampler’s scale (a fixed small sample finalizes among millions of validators) and *adds* a deterministic post-quantum export certificate — without paying BFT’s quadratic message cost or Nakamoto’s latency. The full cross-system comparison is LP-306 [2].

5 ZAP-native transport — no gRPC, no round machine

Nova’s messages ride the ZAP wire (luxfi/zap v0.8.1, a direct dependency of consensus v1.36.0; LP-022 / LP-200–204). ZAP is the single Lux plugin transport: the engine’s Transport[ID] interface (engine/chain/engine.go) is satisfied by the ZAP plugin, and there is **no gRPC plugin server, no Tendermint p2p, and no BFT round/gossip state machine** anywhere on the acceptance path. This matters for the ladder in three ways:

- **Sampling, not broadcast.** Photon queries a k -subset per round; the transport carries $O(k)$ point-to-point polls, not an $O(n^2)$ all-to-all vote. The transport cost of a round is independent of n .
- **Post-quantum by construction.** ZAP frames carry the Lux PQ suite, so a Quasar attestation is a signed ZAP message; the certificate is an aggregate of ZAP-carried signatures. No transport translation layer sits between the signature and the wire.
- **No transport-level view or leader.** Because there is no BFT round machine, there is no view-change, no leader-election message class, and no lock/precommit traffic to carry — the wire has nothing to say about “rounds” because the decider has no rounds.

6 Size-invariance: the same rule from $n = 1$ to $n = 10^6$

K is the **sample** size, not the validator-set size — the load-bearing distinction. For $n \leq K$ a node polls the whole set ($k = n$); for $n > K$ it polls a fixed k -subset each round. The *rule* — repeated k -sampling, α -of- k threshold, β -confidence, accept at the Nova majority, export at the Quasar supermajority — is identical at every scale, and the per-round cost stays $O(k)$ per node regardless of n .

Deployment	K	α	β	Source
Primary-network fleet (live)	5	4	2	node/chains/quorum.go live sizing
DefaultParams	20	14	14	config/config.go
MainnetParams (floor)	21	15	—	config/config.go
Testnet	11	8	—	config/config.go

At the live operating point $K=5, \alpha=4$ the sampler’s per-round agreement bar is $\alpha/K = 80\%$, comfortably clearing the BFT-overlap bound $2\alpha - K = 3 \geq f + 1 = 2$. Note the two thresholds this paper separates are visible here as *distinct knobs*: the sampler’s per-round α (how many of the k polled must agree to score a round) is not the Nova acceptance count (a majority of the set) and is not the Quasar export floor ($> \frac{2}{3}$ stake). What an operator changes as n grows is only which small k to pin; the ladder never changes.

This is exactly the property that a “ $\frac{2}{3}$ -of-*all*-stake certificate decides” model destroys: collecting a supermajority of *every* validator’s signature per height is $O(n)$ work on the critical path. Nova restores size-invariance by keeping the decider a fixed-sample metastable process, and keeps the $\frac{2}{3}$ -stake artifact — but moves it *off* the critical path, onto the Quasar export rung.

7 Empirical evaluation

Environments. Two environments, named once here and referenced by tag:

- **Cluster (M):** the live Lux *testnet*, 5 validators luxd-0..4 running ghcr.io/luxfi/node:v1.36.0 (consensus v1.36.0, evm v1.104.8), namespace lux-testnet on do-sfo3-lux-k8s, C-Chain id 96368, internal RPC /v1/bc/C/rpc port 9640. Measured 2026-07-10.

- **Host (M)**: Apple M1 Max, 10 cores, 64 GB, macOS 26.5.1, go 1.26.4, arm64. Used for the in-process consensus-engine microbenchmarks.

7.1 Coherence under a node-drop — the two-tier property, live

The one property the two-tier ladder exists to deliver is: *a validator falls behind, the chain keeps finalizing locally, the validator rejoins, and no fork ever appears*. We drove a five-validator testnet under steady load (24 in-cluster senders) and scaled the validator set $5 \rightarrow 4 \rightarrow 3 \rightarrow 4 \rightarrow 5$, measuring at each phase the Nova (latest) height advance and *common-height coherence* — all live nodes must hold the *same* block hash at a confirmed height (height skew is not a fork; a hash disagreement is). Harness: `bench/vs-ava/` (`nova_proof.sh`, `nova_loadgen.py`).

Phase	Nova advance (blocks / 70 s)	Coherent (1 hash @ conf.)	Tier	Note
5/5 baseline	+20	yes	Nova + Quasar	both tiers advance
4/5 drop luxd-4	+8	yes	Nova + Quasar	$4/5 \geq \frac{2}{3}$: export still forms
3/5 drop luxd-3	+6	yes	Nova only	majority holds; $< \frac{2}{3}$ so export
4/5 restore luxd-3	(rejoin)	yes	Nova + Quasar	luxd-3 caught up $150 \rightarrow 161$, conv
5/5 restore luxd-4	(rejoin)	yes	Nova + Quasar	full set restored, coherent

All phases held `distinct@conf = 1` (a single common-height hash across live nodes) and `clamp_ok` (the exported/finalized tag never exceeded the latest height). The load-bearing row is **3/5**: the majority tier keeps producing and stays coherent at exactly the point a $\frac{2}{3}$ -only engine freezes ($\lceil 2 \cdot 5/3 \rceil = 4 > 3$), while the export tier *honestly pauses* because no $> \frac{2}{3}$ -stake certificate can form on three of five. The dropped validator rejoined and converged to the common chain (observed luxd-3: $150 \rightarrow 161$). This is Nova (majority, §2) and Quasar ($\frac{2}{3}$ -stake, §3) doing precisely their separate jobs, live. **M**

7.2 Live C-Chain throughput — the ceiling is the fee/gas-target, not consensus

To locate the live throughput ceiling we ran two in-cluster loads against the same five-validator testnet over ~ 90 s windows: a *closed-loop* run (each account advances one nonce only after the prior tx is accepted; `bench/vs-ava/tp_loadgen.py`) and an *open-loop*, *saturating* run (a deep per-account nonce pipeline floods the mempool so the block builder is the only throttle; `tp_loadgen_open.py`). The result is unambiguous under *both* loads: the chain mines a low transaction rate at *low block fullness with the base fee pinned at its floor* — so the binding constraint is the EVM’s dynamic-fee / gas-target throttle, not consensus, which stayed fully coherent.

Metric	closed-loop	open-loop (sat.)	Tag
window / blocks	91 s / 18	90 s / 21	M
mined throughput (tx/s)	1.81	5.72	M
mean block interval	4.38 s	4.15 s	M
mean tx / block	9.17	24.6 (peak 106)	M
block fullness	1.60%	4.31% mean, 18.55% peak	M
base fee	25 gwei floor	25 gwei floor	M
cross-node coherence	yes (all 5 = 0x5960dc41...)	yes (all 5 = 0x9fb2b542...)	M

Even when *saturated* (open-loop), blocks peak at only $\sim 19\%$ full with the base fee still at its 25 gwei floor, and consensus stays coherent — so the chain is nowhere near execution-bound or

consensus-bound; blocks are throttled by the coreth/evm gas-target and block-cadence policy, not by the sampler and not by the EVM’s execution capacity. Consensus was coherent and idle throughout (identical hashes across all five nodes). This reproduces, on v1.36, the prior in-cluster finding that the C-Chain’s live mined ceiling is a *fee-policy* ceiling: consensus absorbs offered load coherently, and the EVM fee/gas-target caps what is mined [5] (M*). It is the honest frame for the head-to-head against *avalanchego* in §7.3: both C-Chains run the same sequential, fee-throttled, geth-derived EVM, so their *live* mined rates are governed by the same throttle — the Lux advantage at v1.36 is the consensus-liveness axis (§7.1), not raw live TPS.

7.3 Relationship to execution — Block-STM is downstream and (today) sequential

Execution is orthogonal to the decider and *downstream* of it: Nova accepts, then the EVM executes the accepted block. On the live C-Chain that execution is **100% sequential coreth-lineage EVM** — the parallel GPU-native Block-STM executor is built and benchmarked but *not wired into the production path* (`RegisterExecutor` has zero callers; `DefaultExecutor()` is a no-op fallback; `evm/core/state_processor.go` falls through to the sequential loop). This is the same execution model *avalanchego*’s C-Chain runs, and the honest reason the two chains’ *live* throughput is comparable. The measured parallel-executor headroom (CPU copy-bound today; projected on GPU) and the standalone *avalanchego* head-to-head are reported in the companion benchmark study (`bench/vs-ava`) and the GPU-native execution paper [4]; consensus does not gate any of it.

7.4 Consensus engine bookkeeping is not the bottleneck

The in-process chain-engine microbenchmarks (`engine/chain/benchmark_test.go`, Host M, median of 5) confirm that block/vote/finalization *bookkeeping* runs orders of magnitude above anything the fee-throttled EVM asks of it:

Operation	median	rate
finalization check	19 ns	~52 M/s
vote processing	424 ns	~2.4 M/s
block acceptance	251 ns	~4.0 M/s
block ingestion	1.2 μ s	~0.83 M/s

Every engine operation is sub-microsecond to low-microsecond — six-plus orders of magnitude above the ~ 1.8 tx/s and ~ 0.2 blk/s the fee-throttled live chain produces (§7.2). The engine’s data-structure throughput is therefore never the live ceiling. (This microbenchmark exercises the engine’s bookkeeping harness, not the full Nova quorum decision, and is reported as a lower bound on engine headroom, not as the Nova finality rate. Raw: `bench/vs-ava/results/engine_bench_raw.txt`)

8 Terminology reconciliation and relationship to prior LPs

“Nova” has been used three ways in the Lux corpus; v1.36 fixes the meaning to the one in the shipped code, and this section states the reconciliation so no reader conflates them.

- **v1.36 Nova (this paper, canonical).** A *rung* on the finality ladder — majority local-acceptance — and, by extension, the *linear-chain consensus mode* that climbs that ladder (`protocol/nova`, `engine/chain/finality.go`). This is the meaning in `consensus v1.36.0` and in LP-134’s taxonomy (Nova = linear mode; Nebula = DAG mode).

- **LP-020 “Nova”** named the linear-chain *engine* (as opposed to Nebula the DAG engine); its component vocabulary (Photon / Wave / Focus / Prism / Horizon / Flare) and its QuasarCert wire survive here *as the attestation layer*. Consistent with this paper once one reads “Nova” as “linear-chain mode.”
- **The earlier “supernova-burst” DAG Nova** (a chapter of *lux-consensus-engines*) described a DAG-frontier burst finalizer. That is a *Nebula*-family (DAG-mode) design; it is *not* the v1.36 linear-chain two-tier ladder specified here, and where the two disagree on what the “Nova” rung means, this paper and LP-305 govern for the shipped linear-chain C-Chain.

Relationship to the specifications:

- **LP-305** [1] is the normative spec of record for this ladder; this paper is its rigorous, measured companion.
- **LP-306** [2] is the cross-system landscape (Lux vs Tendermint / Avalanche / Nakamoto / Gasper / Sui / Aptos / Solana).
- **LP-105** remains the authoritative *lexicon* (public vocabulary and internal names); its finality *state-machine* description predates Nova and is refined by this ladder — read the Nova/Quasar tiers here for the mechanism, LP-105 for the names.
- **LP-010** / **LP-210** specify Block-STM (the execution layer, §7.3); **LP-303** the bounded re-poll / catch-up that heals a paused chain; **LP-217** the cert profile modes (§3).

9 Conclusion

Nova’s contribution is not a new voting rule; it is a *decomposition*. By splitting finality into a majority *local-accept* tier and a $\frac{2}{3}$ -stake *export* tier — and forbidding, by a one-line invariant, any export below the supermajority — Lux v1.36 gets crash-fault liveness (three of five keep producing) *and* Byzantine-safe external finality (nothing leaves the chain without $> \frac{2}{3}$ of stake), without the false choice a single threshold forces. The decider stays a fixed-sample metastable process, so the rule is size-invariant from one validator to a million; the $\frac{2}{3}$ -stake certificate stays post-quantum and portable, but rides the upper rung, off the critical path. The live testnet shows the two tiers doing their separate jobs: coherence held through a $5/5 \rightarrow 4/5 \rightarrow 3/5$ drop while the export tier honestly paused, and the live throughput ceiling sat in the EVM fee policy, not in consensus. The ladder is the spec; the code is the proof.

References

- [1] Lux Industries. *LP-305: Lux Consensus — Canonical Specification (Nova · Quasar · Block-STM)*. <https://github.com/luxfi/lps>.
- [2] Lux Industries. *LP-306: Consensus Comparison — Lux (Nova/Quasar) vs Tendermint, Avalanche, Nakamoto, Gasper, Sui, Aptos, Solana*. <https://github.com/luxfi/lps>.
- [3] Lux Industries. *LP-303: Bounded Re-poll and Runtime Ancestor Catch-up*. <https://github.com/luxfi/lps>.
- [4] Lux Industries. *The Path to One Billion Trades per Second: GPU-Native Execution on Lux*. Lux Papers, 2026.
- [5] Lux Industries. *Lux Quasar Benchmarks and In-Cluster C-Chain Throughput Measurements*. Lux Papers, 2026.

- [6] Team Rocket et al. *Scalable and Probabilistic Leaderless BFT Consensus through Metastability*. 2020. <https://arxiv.org/abs/1906.08936>.
- [7] Gelashvili et al. *Block-STM: Scaling Blockchain Execution by Turning Ordering Curse to a Performance Blessing*. 2022. <https://arxiv.org/abs/2203.06871>.